The Utility Namespaces

Jaxer includes a range of utility functionality grouped into this umbrella namespace, many of which are used by other parts of the framework.

API Links

Jaxer.Util
Jaxer.Util.CRC32
Jaxer.Util.Cookies
Jaxer.Util.DateTime
Jaxer.Util.DOM
Jaxer.Util.MultiHash
Jaxer.Util.Stopwatch
Jaxer.Util.String

Jaxer.Util

- **clone:** clones an object (actually any argument) and returns the clone. If obj is of type "object", then the clone is created from the same constructor (but without any arguments). For a deep clone, every (enumerable) property is itself cloned; otherwise, every (enumerable) property is simply copied (by value or reference).
- **protectedClone:** creates a new object whose private prototype (the one used when looking up property values) will be set to the object passed into this function. This allows for the resulting clone object to add new properties and to redefine property values without affecting the master object. If access to the master object is required, the cloned object contains a `$parent` property which can be used for that purpose.
- **concatArrays:** returns an array whose elements consist of the elements of all the arrays or array-like objects passed in as arguments, with null or undefined elements skipped
- **extend:** extends an object by (shallow) cloning it and then copying all (enumerable) properties from the extensions object to the new cloned object
- **filter:** creates a new array containing the elements of an existing array which meet a function-based criteria
- **filterInPlace:** removes items from an existing array which do not meet a function-based criteria
- **findInGlobalContext:** finds a named object within the global context ('window', in the browser) to which the second argument is ultimately parented.
- **foreach:** applies a function to each element in an array
- **getPropertyNames:** get all, or filtered subset of, the property names from an object as a hash or object
- **hasProperties:** determine if the object contains all properties in a list of names
- **isDate:** tests whether the given object is a Date object (even if it's from a different global context)
- **isEmptyObject:** tests whether the given object is devoid of any (enumerable) properties
- **isNativeFunction:** tests whether the given function is native
- **isNativeFunctionSource:** tests whether the given string is the source of a native function
- **map:** creates a new array by applying the result of a function to each of the items in the array
- **mapInPlace:** replaces each member of an array with the result of a function passed each item as a parameter
- **safeSetValues:** if a property on *values* is not defined on the target object, add that property and associated value to the target object; the target object's [proto] chain is included in the search for each property.
- **sleep:** pauses processing for a specified number of milliseconds

CRC32

CRC32 stands for 32-bit Cyclic redundancy check, generally used as a checksum to detect accidental alteration of data during transmission or storage. There are two ways to get a CRC32 from Jaxer: `getCRC` builds the checksum from an array of bytes and `getStringCRC` builds the checksum from a string.

Using CRC32

```
01. var fl, fc, fca, csarr, csstr;
02. fl = new Jaxer.File('/path/to/your/file.ext');
03. fl.open();
04. fc = fl.read();
05. csstr = Jaxer.Utils.CRC.getStringCRC(fc);
06. fca = fl.readAllLines();
07. csarr = Jaxer.Utils.CRC.getCRC
08.
09. Jaxer.Log.info('CRC32 for ' + fl.path + ' as a string is ' + csstr);
10. Jaxer.Log.info('CRC32 for ' + fl.path + ' as an array of bytes is ' + csarr);
11.
```

Cookies

Cookies are a mainstay of web applications, used to overcome the lack of continuous state presented by browser-based interactions.

- **getAll:** Generates an object containing all cookie keys and values from the current request
- **get:** Retrieves the value of a specified cookie key
- **set:** Store a named value into your cookie
- **parseSetCookieHeaders:** Parse the HTTP response's Set-Cookie string into an array

Using Cookies

```
var rhdrs, phdrs, ckie, ckies;

rhdrs = Jaxer.Response.headers["Set-Cookie"];
phdrs = Jaxer.Utils.Cookies.parseSetCookieHeaders(rhdrs).toString();
ckie1 = Jaxer.Utils.Cookies.get('cookie_name');
Jaxer.Utils.Cookies.set('cookie_name', ckie1 + ' new value');
ckie2 = Jaxer.Utils.Cookies.get('cookie_name');
ckies = Jaxer.Utils.Cookies.getAll().toString();

Jaxer.Log.info('Parsed Set Cookie header is ' + phdrs);
Jaxer.Log.info('Original cookie_name value is ' + ckie1);
Jaxer.Log.info('Changed cookie_name value is ' + ckie2);
Jaxer.Log.info('Flattened value of all cookies is ' + ckies);

```

DateTime

A single convenience method, `toPaddedString`, converts a Date object into a nicely formatted string.

```
var now = new Date;
var pdd = Jaxer.Utils.DateTime.toPaddedString(now);
alert(pdd);
```

DOM

Functions and other objects that extend JavaScript's DOM capabilities, primarily for adding, inserting and replacing script blocks to the document. All the methods (except hashToAttributesString) take a string or array of strings as the contents parameter, which is embedded into the newly created element's innerHTML, and return the new <script>.

- **hashToAttributesString:** Converts an object's properties into an attribute string you can use to create a DOM element
- **createScript:** Makes a new script element based on the attributes passed in
- **replaceWithScript:** Replace a specified element in the DOM with a new script element
- **insertScriptAtBeginning:** Creates a new script element and adds it as the first child of a specified element in the DOM
- **insertScriptBefore:** Creates a new script element and adds it before a specified element in the DOM
- **insertScriptAfter:** Creates a new script element and adds it as the next sibling of the specified element in the DOM
- **insertScriptAtEnd:** Creates a new script element and adds it as the last child of a specified element in the DOM
- **insertHTML:** Injects HTML content into the DOM, this is the HTML equivalent of Jaxer.load and the programmatic equivalent of the <jaxer:include> element but with a bit more control.

Manipulating the DOM

```
// Store the contents of the script block to a variable
var tscript = 'var ts1, ts2;' +
'function testing(v1) {' +
'' +;
'  return v1*v1;' +;
'}' +;
'' +;
'ts1 = testing(22);' +;
'ts2 = testing(33);' +;
'' +;
'alert(\'First test gets \' + ts1);' +;
'alert(\'Second test gets \' + ts2);' +;
'';

var hd = document.getElementById('head');
var atts = new Array();
atts['nm1'] = 11;
atts['nm2'] = 44;
atts['nm3'] = 55;
var as = Jaxer.Util.DOM.hashToAttributesString(atts);
var nscript0 = Jaxer.Util.DOM.createScript(document, tscript, as);
var nscript1 = Jaxer.Util.DOM.insertScriptAtBeginning(document, tscript, hd);
var nscript2 = Jaxer.Util.DOM.insertScriptAtEnd(document, tscript, hd);
var nscript3 = Jaxer.Util.DOM.insertScriptBefore(document, tscript, nscript2);
var nscript4 = Jaxer.Util.DOM.insertScriptAfter(document, tscript, nscript2);

```

Math

Two simple integer-related functions: `forceInteger` converts a string or number into an integer (numbers are rounded down) and `isInteger` checks if a variable stores an integer value.

```
var num1 = 23.333;
var str1 = '23.333';
var isAnInt = Jaxer.Utils.Math.isInteger(num1) ? 'is':'is not';

var msg1 = 'num1 starts as ' + num1 + ' and ' + isAnInt + ' an integer, num1 as an integer is ' +
                Jaxer.Utils.Math.forceInteger(num1);
var msg2 = 'str1 starts as ' + str1 + ' and as an integer is ' +
                Jaxer.Utils.Math.forceInteger(str1);
```

MultiHash

Create and manipulate a hash whose members are primitive values or Arrays of primitive values

- **add:** Adds the name-value pair to the MultiHash, creating the name if it doesn't already exist and converting it to an array if it currently exists as a primitive
- **remove:** Removes the name-value pair from the MultiHash, removing the name entirely if the existing value is a primitive, stepping an array of values to a primitive if only one value remains or simply removing the specified value from an array of values when multiple values remain
- **diff:** Returns the difference between two MultiHash objects

Using a MultiHash

```
var mh1, mh2, df;
mh1 = new Array();
Jaxer.Util.MultiHash.add(mh1, 'nm1', [1,2,3,4,5]);
Jaxer.Util.MultiHash.add(mh1, 'nm2', [21,22,23,24,25]);
Jaxer.Util.MultiHash.add(mh1, 'nm3', 31,32,33,34,35]);
mh2 = new Array();
Jaxer.Util.MultiHash.add(mh2, 'nm1', [1,2,3,4,5]);
Jaxer.Util.MultiHash.add(mh2, 'nm2', [211,122,213,124,251]);
Jaxer.Util.MultiHash.add(mh2, 'nm3', [331,332,333,334,353]);
Jaxer.Util.MultiHash.add(mh2, 'nm4', [431,342,334,434,354]);
Jaxer.Util.MultiHash.remove(mh2, 'nm3', [354]);

df = Jaxer.Util.MultiHash.diff(mh1,mh2);

Jaxer.Log.info('Diff of mh1 and mh2 is ' + df.toString());

```

Stopwatch

Sophisticated timing functionality:

- **clocks:** A hash of named clocks
- **laps:** A hash of named laps
- **timings:** A hash containing all current timers
- **flush:** Gets the current contents of all timers; writes them to the log or passes them to a callback you specify
- **lap:** Adds an element to the laps array for a given timer, the element's value is the difference of the current time and the start of the lap
- **lapcount:** The number of laps for a given timer
- **reset:** Destroys all current timers and clocks
- **start:** Starts a clock
- **stop:** Stops a clock and adds the elapsed time to the timer with the same label—the elapsed time is independent of any lap calls which may have been made since starting the clock

Profiling Your Code

```
var clock, workLap1, workLap2;
Jaxer.Util.Stopwatch.start('masterClock');
// run your setup code
// start the lap timer
Jaxer.Util.Stopwatch.lap('workTimer');
// run the code to be profiled
// mark the first 'lap'
Jaxer.Util.Stopwatch.lap('workTimer');
// capture the first lap time
workLap1 = Jaxer.Util.Stopwatch.laps('workTimer');
// run the code to be profiled again
// mark the second 'lap'
Jaxer.Util.Stopwatch.lap('workTimer');
// capture the second lap time
workLap1 = Jaxer.Util.Stopwatch.laps('workTimer');
// run the teardown code
Jaxer.Util.Stopwatch.stop('masterClock');
// get the overall task time
clock = Jaxer.Util.Stopwatch.clocks.masterClock;

// capture the results
var msg = 'Overall Task Time: ' + clock + '\nFirst Lap: ' +
       workLap1 + '\nSecond Lap: ' + workLap2;

// write the data to the log too
Jaxer.Util.Stopwatch.flush();

// log the results
Jaxer.Log.info(msg);

```

String

Extensions to JavaScript's string capabilities.

Important Note: Neither escapeForJS nor escapeForSQL provide complete protection for your application against XSS, XSRF and other malicious input attacks!

- **grep:** Regular expression matching against a string
- **escapeForJS:** Prepares a string for use in a JavaScript statement
- **escapeForSQL:** Prepares a string for use in a SQL statement
- **singleQuote:** Wraps a string inside single quotes
- **startsWith:** Determines if a string starts with another string, with or without cases matching
- **endsWith:** Determines if a string ends with another string, with or without cases matching
- **trim:** Removes a character or characters of your choice (whitespace by default) from the beginning, end or both of a string
- **upperCaseToCamelCase:**

Grep a File for Links

```
01. var p = '/tmp/foo.dat';
02. var f = new Jaxer.File(p);
03. var lines = f.readAllLines();
04. var links = new Array;
05. var reg = '^(http|https):\/\/[a-z0-9]+([\-\.]{1}[a-z0-9]+)*\.[a-z]{2,5}(:[0-9]{1,5})?(\/.*)?
    ↵ $/ix';
06. links = Jaxer.String.grep(lines, reg);
07. Jaxer.Util.foreach(links, function(curItem, index) {
08.     Jaxer.Log.info('Link ' + index + ': ' + curItem);
09. });
10.
```